automatically generating graphs. For example, a site map of a website is often displayed graphically using nodes and edges. It is tedious to update the site map every time we make changes to the website. It is best to generate it automatically. Graphviz is suitable for such tasks.

Introducing Graphviz

Graphviz is an open source software (licensed under the Common Public License v 1.0) for creating graphs. *Dot* is the language for writing Graphviz programs. Wait—"language for writing programs"? Yes, indeed it is! Graphviz uses declarative programming, and we write programs in plain text, and feed them to the *dot* program to create diagrams. We'll do an example program, and it will become clear in a moment. (See the Resources section for information on how to install Graphviz.)

'Hello world' in dot

Here's how we write a 'hello world' program in *dot*:

```
// hello.dot file
digraph HelloWorld {
        hello -> world
}
```

'Digraph' means a 'directed graph'—a graph with arrows. With *hello -> world* we state that we want to create nodes with the names 'hello' and 'world', connected by an arrow.

We compile the *dot* program and create output thus:

```
$ dot -Tjpeg hello.dot -o hello.jpeg
```

We use the *-T* option to dot to specify the output format; there are numerous supported output formats, like *ps* (Postscript) and *png/gif* (bitmap graphics). We use *-o* to specify an output file name; if we don't, the output will be printed to the console. See Figure 1 for the image that is output by *dot* in response to the *HelloWorld* program.

Simple, isn't it? But it isn't impressive—that's lots of work to get this simple graph; we could have done this with less effort in an interactive graphics editor. However, remember that we have just started with Graphviz, and there is lots to explore. To show the power of Graphviz, we're going to automatically create class diagrams.

Creating a simple class diagram

For class diagrams, the convention is to use rectangles (boxes) to represent classes, and hollow arrow heads to indicate inheritance. Also, in inheritance, the derived class points to a base class; the arrow will end in the base class, so we need to invert the arrows in inheritance diagrams. Here is our first class hierarchy program:

```
// hier.dot file
digraph Hier {
        node[shape = box]
        edge[arrowtail="empty", dir="back"]
        "Base.class" -> "Deri.class"
}
```

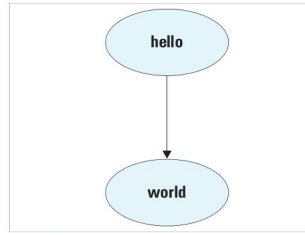

Figure 1: Hello world program output

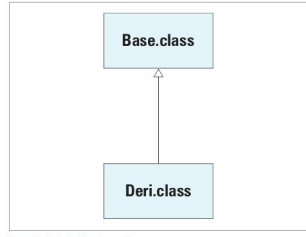

Figure 2: A simple inheritance diagram

We want to show that *Deri.class* inherits from *Base.class*. In a graph, the arrow links are called *edges*, and the end points (like circles, ellipses) are called *nodes*. We can change the *attributes* of such nodes and edges, either globally, or just for a specific edge/node. We want rectangular nodes, so we say *node[shape = box]* to tell the *dot* program this. Similarly, with *edge[arrowtail="empty", dir="back"]* we say that all edges are to have the arrow-tails empty (hollow) and the arrows are to be in the reverse direction.

Why have we used double quotes for *"Deri.class"* and *"Base.class"*? The "." has a special meaning in *dot* language, so, to make sure that *dot* treats *"Deri.class"* as a label of the node, we put it within double quotes. If you compile this sample with *dot*, you'll get the image shown in Figure 2.

Next, we'll use Java to create the diagrams automatically. I assume you know the basics of Java. We're going to use a feature called reflection; in case you're unfamiliar with it, I'll explain how to write simple programs using reflection.